

Zadanie szkoleniowe: Conv1D do regresji

Budowa prostej sieci neuronowej z warstwą Conv1D na zbiorze Pima Indians Diabetes

Motyw przewodni

Zwykle zbiór Pima Indians Diabetes jest używany do klasyfikacji. W tym ćwiczeniu robimy świadomy skręt dydaktyczny: potraktujemy dane jako problem regresji i zbudujemy model przewidujący poziom glukozy na podstawie pozostałych cech pacjentek. Conv1D będzie tu narzędziem do pokazania, jak sieć może uczyć się lokalnych wzorców między uporządkowanymi cechami tabelarycznymi.

Element	Opis
Czas	60-75 minut
Poziom	Podstawowy plus: uczestnik zna Pythona, pandas i ogólną ideę sieci neuronowej
Biblioteki	pandas, numpy, scikit-learn, tensorflow/keras, matplotlib
Typ problemu	Regresja: predykcja wartości ciągłej
Architektura	Conv1D + pooling + warstwy Dense
Efekt końcowy	Działający notebook/skrypt, metryki MAE/RMSE oraz krótka interpretacja wyników

1. Cel ćwiczenia

- Zrozumieć, jak przygotować dane tabelaryczne do wejścia warstwy Conv1D.
- Zbudować prostą sieć neuronową do regresji w Keras/TensorFlow.
- Porównać błąd modelu na danych treningowych i testowych.
- Zobaczyć, że Conv1D nie jest zarezerwowane wyłącznie dla tekstu i szeregów czasowych, choć w danych tabelarycznych trzeba ostrożnie interpretować sąsiedztwo cech.
- Nauczyć się oceniać model regresyjny za pomocą MAE, MSE, RMSE i wykresu wartości rzeczywistych kontra przewidywanych.

2. Opis problemu biznesowo-analitycznego

Scenariusz: zespół analityczny chce przygotować prosty model, który szacuje poziom glukozy u pacjentek na podstawie innych cech medycznych. Model nie ma być narzędziem diagnostycznym. To ćwiczenie służy wyłącznie do nauki architektury sieci neuronowej i procesu uczenia regresji.

Uwaga etyczna i metodologiczna

Nie interpretujemy wyniku jako diagnozy. Dane medyczne wymagają walidacji klinicznej, kontroli biasu, zgód, bezpieczeństwa i audytu. Tu budujemy miniaturowy poligon treningowy, nie elektronicznego lekarza w pudełku.

3. Dane wejściowe

Zbiór Pima Indians Diabetes zawiera 768 obserwacji i 9 kolumn. W klasycznej wersji zmienna *Outcome* jest etykietą klasyfikacyjną, ale w tym zadaniu nie trenujemy klasyfikatora. Zmienną docelową będzie *Glucose*, czyli wartość ciągła.

Kolumna	Znaczenie w ćwiczeniu
Pregnancies	Liczba ciąż, cecha wejściowa
Glucose	Poziom glukozy, zmienna docelowa regresji
BloodPressure	Ciśnienie krwi, cecha wejściowa
SkinThickness	Grubość fałdu skórniego, cecha wejściowa
Insulin	Poziom insuliny, cecha wejściowa
BMI	Wskaźnik masy ciała, cecha wejściowa
DiabetesPedigreeFunction	Funkcja obciążenia rodzinnego, cecha wejściowa
Age	Wiek, cecha wejściowa
Outcome	Klasyczna etykieta klasyfikacyjna. W wersji podstawowej można ją pominąć albo użyć wyłącznie w eksperymencie dodatkowym.

4. Zadanie dla uczestników

Zbuduj model regresyjny oparty o warstwę Conv1D, który przewiduje wartość kolumny *Glucose*. Przejdź przez pełny mini-pipeline ML: wczytanie danych, oczyszczenie, skalowanie, przygotowanie tensora 3D, trening modelu, ewaluacja oraz krótka interpretacja.

Kroki obowiązkowe

1. Wczytaj dane z pliku CSV i nadaj kolumnom czytelne nazwy.
2. Sprawdź rozmiar danych, typy kolumn i podstawowe statystyki.
3. Zamień wartości 0 na braki danych w kolumnach, w których zero jest nielogiczne medycznie: *Glucose*, *BloodPressure*, *SkinThickness*, *Insulin*, *BMI*.
4. Usuń obserwacje bez wartości docelowej *Glucose*, a braki w cechach wejściowych uzupełnij medianą wyliczoną na zbiorze treningowym.
5. Zbuduj macierz cech *X* oraz wektor celu *y*.
6. Podziel dane na train/test w proporcji 80/20.
7. Przeskaluj cechy i zmienną docelową.
8. Przekształć *X* do formatu wymaganego przez Conv1D: liczba_próbek x liczba_cech x 1.
9. Zbuduj, skompiluj i wytrenuj sieć Conv1D.
10. Policz MAE i RMSE na zbiorze testowym.
11. Narysuj wykres strat oraz wykres *y_true* kontra *y_pred*.
12. Napisz 3-5 zdań interpretacji: czy model działa sensownie, gdzie może się mylić i dlaczego Conv1D w tym zadaniu jest raczej demonstracją architektury niż oczywistym wyborem produkcyjnym.

5. Kod startowy dla uczestników

Poniższy kod jest szkieletem. Uczestnicy uzupełniają miejsca oznaczone komentarzem *TODO*. Prowadzący może wcześniej przygotować plik CSV jako *pima.csv* w katalogu roboczym notebooka.

```
import numpy as np
import pandas as pd
```

```

import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

import tensorflow as tf
from tensorflow.keras import layers, models

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

columns = [
    "Pregnancies", "Glucose", "BloodPressure", "SkinThickness",
    "Insulin", "BMI", "DiabetesPedigreeFunction", "Age", "Outcome"
]

# TODO 1: wczytaj dane z pliku pima.csv

print(df.shape)
print(df.head())
print(df.describe().T)

# W tych kolumnach zero traktujemy jako potencjalny brak danych.
zero_as_missing = [
    "Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"
]
df[zero_as_missing] = df[zero_as_missing].replace(0, np.nan)

# Problem regresji: przewidujemy Glucose.
target_col = "Glucose"
feature_cols = [
    "Pregnancies", "BloodPressure", "SkinThickness", "Insulin",
    "BMI", "DiabetesPedigreeFunction", "Age"
]

# Usuwamy wiersze bez wartości docelowej.
df_model = df.dropna(subset=[target_col]).copy()

X = df_model[feature_cols]
y = df_model[target_col].to_numpy().reshape(-1, 1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED
)

# Pipeline dla X: imputacja medianą + standaryzacja.
x_preprocess = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

X_train_scaled = x_preprocess.fit_transform(X_train)
X_test_scaled = x_preprocess.transform(X_test)

# Skalujemy y, żeby sieć uczyła się stabilniej.
y_scaler = StandardScaler()
y_train_scaled = y_scaler.fit_transform(y_train)
y_test_scaled = y_scaler.transform(y_test)

# Conv1D oczekuje tensora 3D: próbki, kroki, kanały.

```

```

X_train_conv = X_train_scaled[..., np.newaxis]
X_test_conv = X_test_scaled[..., np.newaxis]

print("X_train_conv shape:", X_train_conv.shape)
print("X_test_conv shape:", X_test_conv.shape)

# TODO 2: zbuduj model Conv1D.
model.summary()

# TODO 3: uruchom trening.

# Ewaluacja.
y_pred_scaled = model.predict(X_test_conv)
y_pred = y_scaler.inverse_transform(y_pred_scaled)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"MAE: {mae:.2f}")
print(f"RMSE: {rmse:.2f}")

# Wykres straty.
plt.figure(figsize=(8, 4))
plt.plot(history.history["loss"], label="train loss")
plt.plot(history.history["val_loss"], label="validation loss")
plt.xlabel("Epoka")
plt.ylabel("MSE na skali standaryzowanej")
plt.legend()
plt.title("Krzywa uczenia")
plt.show()

# Wykres wartości rzeczywistych kontra predykcje.
plt.figure(figsize=(5, 5))
plt.scatter(y_test, y_pred, alpha=0.7)
plt.xlabel("Glucose - wartość rzeczywista")
plt.ylabel("Glucose - predykcja")
plt.title("Predykcje modelu Conv1D")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()])
plt.show()

```

6. Wyjaśnienie architektury

Warstwa	Rola
Input	Przyjmuje tensor o kształcie: liczba cech x 1. Każda cecha jest traktowana jak kolejny krok krótkiej sekwencji.
Conv1D	Uczy małych lokalnych filtrów. Filtr o kernel_size=2 patrzy na pary sąsiednich cech w ustalonej kolejności.
Conv1D	Druga warstwa buduje bardziej złożone reprezentacje na podstawie pierwszych filtrów.
GlobalAveragePooling1D	Redukuje wymiar sekwencyjny i zamienia mapy cech na wektor.
Dense	Łączy wyuczone reprezentacje i wykonuje nieliniową transformację.
Dense(1)	Zwraca jedną liczbę: przewidywaną wartość Glucose na skali standaryzowanej.

Ważna interpretacja Conv1D w danych tabelarycznych

Conv1D zakłada pewną lokalność: sąsiednie pozycje mają znaczenie. W szeregach czasowych to naturalne, bo sąsiednie punkty leżą obok siebie w czasie. W tabeli medycznej kolejność cech jest sztuczna. Dlatego to ćwiczenie jest świetne do nauki mechaniki Conv1D, ale w prawdziwym projekcie trzeba porównać je z prostszym modelem Dense, Random Forest lub Gradient Boosting.

7. Zadania dodatkowe dla szybszych grup

1. Zmień target z Glucose na BMI. Czy model uczy się łatwiej czy trudniej?
2. Dodaj kolumnę Outcome jako cechę wejściową i sprawdź, czy wynik regresji się poprawia. Omów ryzyko interpretacyjne takiego wyboru.
3. Porównaj Conv1D z prostą siecią Dense bez konwolucji.
4. Zmień kernel_size z 2 na 3. Co dzieje się z MAE i RMSE?
5. Zmień kolejność cech wejściowych. Czy wynik modelu jest stabilny? Co to mówi o Conv1D na danych tabelarycznych?
6. Dodaj Dropout lub BatchNormalization i porównaj krzywe uczenia.

8. Mini-wariant porównawczy: Dense baseline

To krótki model kontrolny. Celem nie jest pobicie Conv1D za wszelką cenę, tylko sprawdzenie, czy konwolucja rzeczywiście daje przewagę na tak małym zbiorze.

```
dense_model = models.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1],)),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1)
])

dense_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="mse",
    metrics=[tf.keras.metrics.MeanAbsoluteError(name="mae")]
)

dense_history = dense_model.fit(
    X_train_scaled,
    y_train_scaled,
    validation_split=0.2,
    epochs=200,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)

y_pred_dense_scaled = dense_model.predict(X_test_scaled)
y_pred_dense = y_scaler.inverse_transform(y_pred_dense_scaled)

mae_dense = mean_absolute_error(y_test, y_pred_dense)
rmse_dense = np.sqrt(mean_squared_error(y_test, y_pred_dense))

print(f"Dense MAE: {mae_dense:.2f}")
print(f"Dense RMSE: {rmse_dense:.2f}")
```

9. Pytania kontrolne

1. Dlaczego X dla Conv1D musi mieć trzy wymiary?
2. Czym różni się loss MSE od metryki MAE?
3. Dlaczego skalujemy zmienną docelową y w regresji neuronowej?
4. Dlaczego wartości 0 w niektórych kolumnach Pima mogą oznaczać brak danych?
5. Dlaczego Conv1D na danych tabelarycznych wymaga ostrożnej interpretacji?
6. Co oznacza overfitting na wykresie loss/val_loss?
7. Kiedy użyłbyś prostego modelu Dense zamiast Conv1D?